PROBLEM 1: INVENTORY MANAGEMENT SYSTEM - PRODUCTS API SOLUTION SUMMARY

1. Problem:
The original API created product and inventory separately with multiple commits, no validation, and no error handling, leading to inconsistent data in production.

2. Key Issues:
- No input validation
- No SKU uniqueness handling
- Multiple DB commits (partial writes possible)
- No transaction safety
- No price validation
- No error handling

3. Solution Implemented:
- Added JSON validation
- Validated required fields
- Ensured price is decimal and non-negative
- Validated initial_quantity (integer, non-negative)
- Used db.session.flush() + single commit for atomicity
- Handled duplicate SKU using IntegrityError
- Added proper error handling and responses

4. Design Decisions:
- Separated Product and Inventory tables
- Supports multiple warehouses per product
- Used Decimal for price to avoid precision issues
- Used DB constraints for consistency

5. Edge Cases Covered:
- Missing fields
- Invalid JSON
- Invalid/negative price
- Invalid/negative quantity
- Duplicate SKU
- Partial failure prevention

6. Assumptions:
- SKU is globally unique
- initial_quantity defaults to 0
- warehouse_id is valid (out of scope)
- Price stored as Decimal

7. Improvements (Future Scope):
- Validate warehouse_id
- Add idempotency
- Add service layer
- Better logging & monitoring

PROBLEM 2 : DATABASE DESIGN - PART 2 (GAPS + DECISIONS)

1. Identify Gaps (Questions to ask product team)

- is SKU unique globally or per company?
- can one product belong to multiple companies or only one?
- can warehouses store products from different companies?
- do we need to track who updated inventory (user/admin)?
- what are all possible inventory change types (only add/remove/update or more)?
- do we need to track reserved stock or damaged stock?
- can suppliers give different prices for same product over time?
- should supplier price history be stored?
- can bundles contain other bundles (nested bundles)?
- should bundle price be calculated or stored separately?
- what happens if a product in bundle is deleted?
- do we need soft delete for products/warehouses?
- should inventory updates be real-time or batch-based?

2. Explain Design Decisions

- separated products and inventory tables to support multiple warehouses per product
- used UNIQUE(product_id, warehouse_id) to avoid duplicate inventory rows
- used separate inventory_logs table to track stock changes for audit/debug
- used DECIMAL for price to avoid floating point issues
- used many-to-many table (supplier_products) for supplier-product relation
- used self-referencing table (product_bundles) to handle bundle products
- kept SKU unique to maintain product identity across system
- added foreign keys to maintain data integrity between tables
- timestamps added to track record creation and updates

Indexes (for performance):
- index on products(sku) for fast lookup
- index on inventory(product_id) for product queries
- index on inventory(warehouse_id) for warehouse queries

PROBLEM 3 : LOW STOCK ALERT API - PART 3 SUMMARY

1. Problem:
Build an API to return low stock alerts for a company considering:
- product-specific thresholds
- recent sales activity
- multiple warehouses
- supplier info for reordering

2. Implementation Approach:
- Fetch products for the given company
- Filter products with recent sales (assumed last 30 days)
- For each product, check inventory across all warehouses
- Compare current stock with threshold
- If stock is low, calculate days_until_stockout using average daily sales
- Fetch supplier info and include it in response
- Return alerts list with total count

3. Assumptions:
- Sales table exists to track product activity
- Recent sales window = 30 days
- low_stock_threshold exists in product (default = 10 if missing)
- One supplier per product (simplified)
- Supplier has contact email
- Simple formula used for stockout prediction

4. Edge Cases Handled:
- No recent sales → product ignored
- No supplier → return null
- Zero sales → avoid division by zero
- Multiple warehouses handled correctly
- Missing threshold → default used

5. Possible Issues:
- N+1 query problem (multiple DB calls)
- Performance issues with large data
- Incorrect threshold configuration
- Missing or inconsistent sales data

6. Improvements:
- Use JOIN queries instead of loops
- Add pagination for large responses
- Add caching
- Support multiple suppliers
- Improve stock prediction logic